DOCUMENTAÇÃO DE SOFTWARE

Patinhas em Ação

Aplicativo de proteção animal – Arvorezinha/RS

Entrega 3 – Sprint 3: Interação e Social

Univates – Lajeado/RS
Novembro de 2026

Equipe de desenvolvimento do projeto:

Nome	Função
Arthur Predebon Rostirolla	Analista/Desenvolvedor
João Alberto Gehlen	Desenvolvedor/Documentador
Pedro Oliveira Fonseca	Desenvolvedor/Testador

Histórico de versões

Versão	Data	Descrição
1.0	14/09/2026	Versão inicial: requisitos, modelagem, arquitetura, tecnologias, backlog e planejamento das sprints.
1.1	05/10/2026	Entrega 2: Sprint 2 concluída, com a denúncia sem conexão antecipada da Sprint 3; ajustes em RF0011, RF0014, RF0019 e RNF0002; modelo de dados, tecnologias e backlog atualizados; planejamento da Sprint 3.
1.2	09/11/2026	Entrega 3: Sprint 3 concluída (mural de notícias, formulário de adoção, foto de perfil e central de avisos), atualização para o Expo SDK 58 e refino da interface; ajustes em RF0001, RF0006, RF0012, RF0013, RF0016, RF0017, RNF0007 e RNF0010; diagrama de casos de uso e modelo de dados refeitos; novas tabelas e itens de backlog; validação com a ONG e primeiros registros.

1. Introdução

Área de negócio

O projeto atua na área de proteção e bem-estar animal, em parceria com a ONG Patinhas em Ação, organização que realiza resgates, tratamento e encaminhamento para adoção de cães e gatos abandonados ou em situação de risco. O aplicativo apoia todo o ciclo de proteção: da denúncia feita pela comunidade até a adoção responsável e a prestação de contas das doações recebidas.

Território

O território de atuação é o município de Arvorezinha, no Rio Grande do Sul. Por ser uma cidade de pequeno porte, a comunicação entre moradores, voluntários e a ONG acontece hoje de forma informal, principalmente por redes sociais e aplicativos de mensagens.

Problema a ser resolvido

A ONG e a comunidade têm dificuldade em centralizar e monitorar denúncias de animais abandonados ou em risco. A comunicação descentralizada causa perda de informações críticas (localização, fotos, condição do animal), atraso nos resgates e falta de transparência sobre a situação de cada animal. Também não existe um repositório único que facilite a adoção responsável ou a gestão de doações pontuais de insumos e recursos financeiros.

Público-alvo

- **Moradores:** pessoas que desejam denunciar animais em risco, acompanhar os resgates ou adotar.
- **Visitantes sem conta:** qualquer pessoa pode abrir o aplicativo, ver o mapa e registrar uma denúncia sem se cadastrar, acompanhando depois apenas as denúncias que ela mesma abriu.
- **Administradores da ONG:** gestores do catálogo de adoção, das situações de resgate, dos usuários, das campanhas de arrecadação, do mural de notícias e da avaliação dos pedidos de adoção.

Objetivo e escopo

O objetivo do Patinhas em Ação é oferecer um aplicativo móvel que centralize as informações da ONG e dê transparência ao trabalho realizado. O escopo foi dividido em quatro módulos integrados:

- **Denúncias:** registro com foto, descrição e geolocalização automática do animal, inclusive sem conexão.
- **Acompanhamento:** linha do tempo do animal (Denunciado → Resgatado → Tratamento → Adoção), visível ao denunciante, com avisos dentro do aplicativo quando a situação muda.
- **Adoção:** catálogo com filtros de porte, idade e temperamento e formulário digital de adoção avaliado pela ONG.

- **Doações:** campanhas de arrecadação (vaquinhas) com meta, valor já arrecadado e chave PIX copiável. O aplicativo informa e dá transparência; nenhum pagamento é processado por ele.

Esta terceira entrega corresponde à Sprint 3 (Interação e Social): mural de notícias da ONG, formulário de adoção, foto de perfil e central de avisos, além da atualização para o Expo SDK 58 e do refino da interface. A Sprint 1 (Base e Cadastro) entregou o protótipo funcional, a arquitetura, o banco de dados e o cadastro de animais e usuários; a Sprint 2 (Localização e Status), o mapa, o GPS, o fluxo de situação do resgate, a migração para o Supabase, o modo convidado, a denúncia sem conexão e as vaquinhas.

2. Requisitos Funcionais (RFs)

Os requisitos abaixo foram levantados com a ONG Patinhas em Ação e classificados por prioridade e complexidade. A coluna Versão indica a versão deste documento em que o requisito foi registrado ou alterado pela última vez.

RF0001 – Cadastro de usuário		
O sistema deve permitir que qualquer pessoa crie uma conta informando apenas nome completo, e-mail e senha; telefone e foto de perfil são completados depois, em Meus dados, para que o cadastro seja rápido. Toda conta criada pelo autocadastro recebe o perfil Morador. O e-mail deve ser único e os dados devem ser validados, exibindo mensagens de erro por campo. Quando o cadastro é feito a partir de uma sessão de visitante, a conta é criada sobre a mesma sessão, preservando as denúncias já registradas.		
Prioridade	**Complexidade**	**Versão**
Alta	Baixa	1.2

RF0002 – Autenticação		
O sistema deve permitir que o usuário entre com e-mail e senha e saia da conta. A sessão deve ser mantida entre aberturas do aplicativo, e as telas devem ser liberadas de acordo com o perfil do usuário autenticado.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.0

RF0003 – Gerenciamento de usuários		
O administrador deve poder listar, buscar, filtrar por perfil, cadastrar, editar e excluir usuários, definindo o perfil de acesso (Morador ou Administrador). A exclusão é lógica: o perfil deixa de ser visível e perde o acesso, mas os registros que a pessoa criou preservam a autoria. O sistema não deve permitir que o administrador exclua a própria conta nem que fique sem nenhum administrador.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.0

RF0004 – Manutenção de perfil		
O usuário autenticado deve poder editar seus dados pessoais e alterar sua senha, informando a senha atual e a confirmação da nova senha. Apenas administradores podem alterar o perfil de acesso.		
Prioridade	**Complexidade**	**Versão**
Média	Baixa	1.0

RF0005 – Registro de animal e denúncia		
O usuário deve poder registrar um animal com foto (câmera ou galeria), nome, espécie, porte, sexo, idade aproximada, temperamento, descrição e observações de saúde. Registros feitos por moradores e visitantes devem entrar com a situação "Denunciado"; apenas administradores podem definir a situação inicial. O temperamento "Bravo" deve sinalizar que o animal exige cuidado na aproximação.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.0

RF0006 – Catálogo de animais		
O sistema deve exibir os animais cadastrados em forma de catálogo com foto e situação, permitindo busca por nome ou descrição. O filtro por situação deve ficar sempre visível e exibir a contagem de animais em cada uma; espécie e porte devem ficar em um painel recolhível. Sem filtro de situação, o catálogo mostra os animais ativos; os adotados ficam acessíveis pelo filtro "Adotado".		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.2

RF0007 – Edição e exclusão de animais		
Somente administradores devem poder editar e excluir animais, e animais adotados não podem ser excluídos. A edição pelo próprio denunciante não está prevista: o trabalho de campo fica concentrado na ONG.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.0

RF0008 – Histórico do animal		
O sistema deve registrar cada mudança de situação do animal (situação anterior, nova situação, observação, responsável e data) e exibir a linha do tempo e o progresso da jornada na tela de detalhes.		
Prioridade	**Complexidade**	**Versão**
Média	Média	1.0

RF0009 – Geolocalização da denúncia		
Ao registrar uma denúncia, o aplicativo deve capturar automaticamente a localização do dispositivo via GPS assim que o formulário é aberto, e permitir atualizá-la com um toque. A denúncia deve ser aceita sem coordenada, avisando que o animal não aparecerá no mapa.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.0

RF0010 – Mapa de ocorrências		
O mapa deve ser a tela inicial do aplicativo. Deve exibir a localização do usuário e um marcador por animal, na cor da sua situação, com um cartão de foto e nome ao toque. Deve usar Google Maps no Android e Apple Maps no iOS. O traçado de rota até o animal fica fora do escopo desta versão.		
Prioridade	**Complexidade**	**Versão**
Média	Alta	1.0

RF0011 – Alteração da situação do resgate		
O administrador deve avançar a situação do animal seguindo o fluxo Denunciado → Resgatado → Em tratamento → Disponível para adoção → Adotado, oferecendo só as transições válidas. Cada mudança é gravada no histórico pelo banco de dados, com uma observação opcional.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.1

RF0012 – Avisos para a equipe da ONG e para a comunidade		
O sistema deve avisar a equipe da ONG quando uma denúncia nova ou um pedido de adoção for registrado, avisar o denunciante quando a situação do seu animal mudar e avisar quem pediu adoção quando o pedido for avaliado. Os avisos ficam numa central dentro do aplicativo, com contador de não lidos. Notificações push exigem build própria do aplicativo (o Expo Go não as entrega) e ficam para depois.		
Prioridade	**Complexidade**	**Versão**
Média	Alta	1.2

RF0013 – Formulário de adoção		
O morador deve poder pedir a adoção de um animal disponível preenchendo um formulário digital (moradia, pátio, outros animais, crianças, experiência e motivo). O pedido segue Enviado → Em análise → Aprovado ou Recusado, pode ser recusado já de início e é avaliado pela ONG numa fila, com o contato de quem pediu. É permitido um pedido aberto por pessoa e animal. Ao aprovar, o administrador pode marcar o animal como adotado no mesmo passo, o que recusa os demais pedidos abertos para ele.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.2

RF0014 – Necessidades e doações		
Exibir as campanhas abertas com capa, meta, arrecadado e barra de progresso; cada uma tem página com detalhes, fotos, animal vinculado, PIX copiável e histórico. O administrador cria, edita, encerra e exclui campanhas e lança os valores recebidos, somados pelo banco. O app não recebe pagamentos.		
Prioridade	**Complexidade**	**Versão**
Média	Baixa	1.1

RF0015 – Prestação de contas		
O administrador deve poder registrar as doações recebidas e sua aplicação, e o aplicativo deve apresentar esses dados de forma visual para a comunidade.		
Prioridade	**Complexidade**	**Versão**
Média	Média	1.0

RF0016 – Foto de perfil do usuário		
O usuário deve poder definir uma foto de perfil em Meus dados, exibida no seu cadastro e, junto com o nome, ao lado das ações que ele registra no histórico dos animais. Em uma ONG que opera por vínculo pessoal, reconhecer quem atendeu o caso faz parte da confiança na informação.		
Prioridade	**Complexidade**	**Versão**
Baixa	Baixa	1.2

RF0017 – Mural de notícias da ONG		
O administrador deve poder publicar notícias e atualizações curtas — resgates concluídos, campanhas, mutirões — visíveis a todos os usuários com conta. Cada notícia tem título, texto, capa, até seis fotos e vínculo opcional com um animal ou uma vaquinha, e uma delas pode ser fixada no topo. Sustenta o canal performativo identificado por Ghoorah et al. (2025): a percepção de que a organização comunica com regularidade gera confiança, mesmo em quem não lê os números.		
Prioridade	**Complexidade**	**Versão**
Média	Média	1.2

RF0018 – Denúncia sem conexão		
O aplicativo deve permitir registrar uma denúncia sem internet, guardando foto, descrição e coordenada no aparelho e enviando assim que a conexão voltar, com indicação clara do que ainda está pendente. A denúncia acontece na rua, onde o sinal é irregular; sem a fila no aparelho, o registro se perderia.		
Prioridade	**Complexidade**	**Versão**
Alta	Alta	1.1

RF0019 – Cadastro e ocupação de lares temporários		
Fora do escopo: retirado do projeto por decisão da equipe na Entrega 2. O requisito previa registrar os lares temporários disponíveis, quantos animais cada um abriga e desde quando, exibindo a ocupação atual.		
Prioridade	**Complexidade**	**Versão**
Alta	Alta	1.1

RF0020 – Acompanhamento pós-adoção		
Após uma adoção, o sistema deve criar lembretes de contato com o adotante nos primeiros quinze dias e permitir registrar o resultado de cada contato, incluindo devolução. Thumpkin et al. (2022) observaram que dois terços das devoluções acontecem nos primeiros catorze dias: é aí que a adoção se confirma ou se desfaz, e não na divulgação.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.0

RF0021 – Fila de casos abertos		
O sistema deve apresentar à ONG a fila de denúncias ainda não atendidas, ordenada por tempo de espera, com a contagem de casos abertos. Atende ao gap G1 da pesquisa de trabalhos relacionados: hoje os relatos entram por vários canais e ninguém sabe quantos casos estão pendentes.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.0

RF0022 – Registro de acionamento de voluntários		
O sistema deve registrar quem foi acionado para cada caso e tornar visível a distribuição desses acionamentos ao longo do tempo. Kaur et al. (2022) mostram que designar sempre a mesma pessoa consome capacidade futura: a sobrecarga de hoje é o voluntário perdido amanhã. Não se propõe otimização automática — para três voluntários seria desproporcional —, apenas tornar a concentração visível.		
Prioridade	**Complexidade**	**Versão**
Média	Média	1.0

RF0023 – Prestação de contas periódica da operação		
Além das campanhas por caso, o sistema deve permitir publicar um resumo periódico e simples do que entrou e do que foi gasto pela ONG. Ghoorah et al. (2025) indicam que a doação recorrente nasce da confiança na organização, e não da comoção com um animal específico, o que sugere a operação — e não o caso isolado — como unidade de prestação de contas.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.0

3. Requisitos Não-Funcionais (RNFs)

RNF0001 – Plataforma		
O aplicativo deve ser executado em dispositivos Android e iOS a partir de uma única base de código, desenvolvida em React Native com Expo e TypeScript.		
Prioridade	**Complexidade**	**Versão**
Alta	Baixa	1.0

RNF0002 – Persistência dos dados		
Os dados devem ser armazenados em banco PostgreSQL gerenciado pelo Supabase, compartilhado entre todos os usuários: a denúncia registrada por um morador deve ser vista imediatamente pela ONG. Sem conexão, só a denúncia funciona: ela fica no aparelho e é enviada quando a internet voltar (RF0018).		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.1

RNF0003 – Segurança das credenciais		
A autenticação deve ser delegada ao Supabase Auth: nenhuma senha pode ser processada ou armazenada pelo aplicativo. A sessão deve ser mantida no armazenamento local do dispositivo e renovada automaticamente. Visitantes devem usar sessão anônima, convertida em conta definitiva no cadastro.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.0

RNF0004 – Controle de acesso		
As permissões de cada perfil devem ser definidas em um único ponto do aplicativo e verificadas tanto na interface (ocultando ações) quanto nos serviços. A verificação que de fato protege os dados, porém, é a do servidor: políticas de Row Level Security no PostgreSQL devem espelhar essa matriz e recusar qualquer operação não autorizada, inclusive requisições feitas fora do aplicativo.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.0

RNF0005 – Integridade dos dados		
O banco deve utilizar chaves primárias e estrangeiras, tipos enumerados, restrições de domínio (CHECK), índices e gatilhos. Operações que precisam ser atômicas, como o cadastro do animal com a primeira entrada do histórico, devem ser garantidas por gatilhos no servidor. Alterações de estrutura devem ser aplicadas por migrações versionadas e numeradas.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.0

RNF0006 – Manutenibilidade		
O software deve seguir uma arquitetura em camadas (apresentação, aplicação, domínio e infraestrutura), com as dependências apontando sempre para o domínio. O objetivo é que trocar o mecanismo de persistência não obrigue a alterar telas, regras de negócio ou serviços. A autenticação é a exceção conhecida: por não ser persistência, depende de uma porta própria para o provedor externo.		
Prioridade	**Complexidade**	**Versão**
Alta	Média	1.0

RNF0007 – Usabilidade		
A interface deve estar em português, seguir uma identidade visual consistente, exibir mensagens de validação junto aos campos e se adaptar a diferentes tamanhos de tela, respeitando a área segura dos aparelhos e limitando a largura do conteúdo em tablets. As animações devem respeitar a opção de reduzir movimento do sistema.		
Prioridade	**Complexidade**	**Versão**
Média	Média	1.2

RNF0008 – Testabilidade		
As regras de negócio e os serviços devem ser cobertos por testes automatizados executáveis sem emulador, utilizando repositórios em memória.		
Prioridade	**Complexidade**	**Versão**
Média	Baixa	1.0

RNF0009 – Privacidade e permissões		
O acesso à câmera e à galeria deve ser solicitado apenas quando necessário, com mensagem explicando o motivo do uso.		
Prioridade	**Complexidade**	**Versão**
Média	Baixa	1.0

RNF0010 – Sincronização em nuvem		
Os dados e as imagens devem ser armazenados em serviço de nuvem (Supabase: PostgreSQL e Storage), com compartilhamento entre todos os usuários. As fotos são reduzidas no aparelho antes do envio (lado maior de até 1600 px), para economizar dados móveis, espaço no Storage e memória em celulares modestos. O Firebase, citado na proposta inicial, foi descartado em favor do Supabase por oferecer banco relacional, que preserva o modelo de dados projetado.		
Prioridade	**Complexidade**	**Versão**
Média	Alta	1.2

4. Diagrama(s) de Casos de Uso

O diagrama a seguir apresenta os atores e os casos de uso do Patinhas em Ação, de acordo com o padrão UML, agrupados por assunto em quatro pacotes: conta e acesso, denúncias e resgate, adoção, e comunidade e doações. Os atores formam uma hierarquia de generalização: o Morador herda tudo o que o Visitante faz, e o Administrador herda tudo o que o Morador faz. Por isso, cada caso de uso aparece ligado apenas ao ator mais básico que o executa, e a cor indica esse ator: azul para o que todos fazem, verde para o que exige conta e laranja para o que é exclusivo do administrador.

Visitante e Morador ficam à esquerda, ligados à coluna da esquerda; o Administrador fica à direita, ligado à coluna da direita. No centro, tracejados em cinza, estão os casos de uso incluídos ou estendidos: a captura do GPS faz parte da denúncia («include»), e guardar a denúncia sem conexão, lançar um valor numa vaquinha e registrar a adoção ao aprovar um pedido estendem o caso de uso base («extend»). O Visitante — chamado de convidado no aplicativo — usa uma sessão anônima e vê apenas as denúncias que ele mesmo abriu. O caso de uso tracejado em laranja está planejado para a Sprint final.

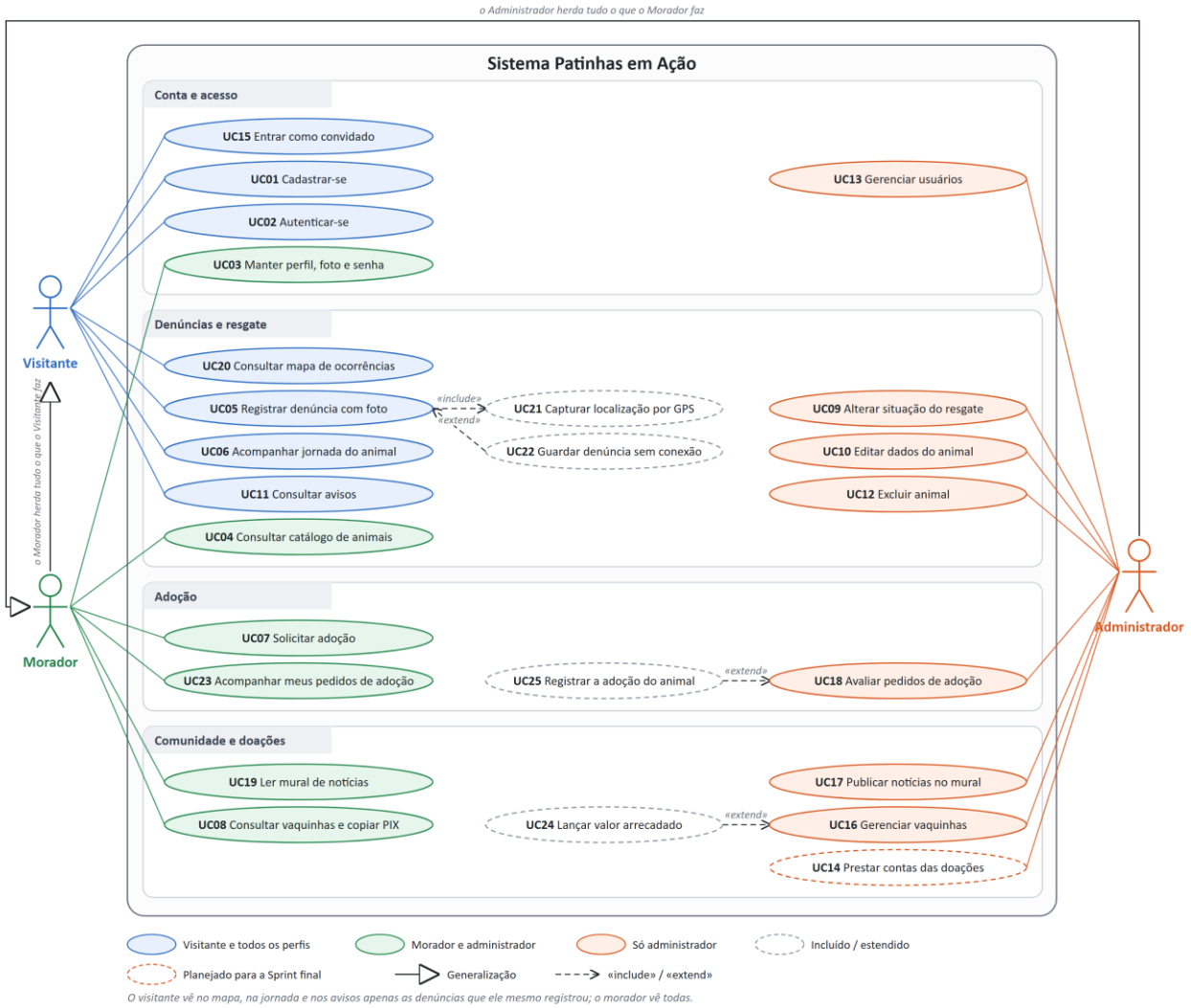


Figura 1 – Diagrama de casos de uso do Patinhas em Ação

Descrição resumida dos casos de uso

A coluna Sprint indica em qual entrega cada caso de uso foi ou será implementado.

ID	Caso de uso	Ator principal	Descrição	Sprint
UC01	Cadastrar-se	Visitante	Cria uma conta com nome, e-mail e senha; a conta nasce como Morador e mantém as denúncias feitas como convidado.	1
UC02	Autenticar-se	Visitante	Entra e sai do aplicativo com e-mail e senha.	1
UC03	Manter perfil, foto e senha	Morador	Edita dados pessoais e foto de perfil e troca a senha.	1
UC04	Consultar catálogo de animais	Morador	Busca e filtra animais por espécie, porte e situação.	1
UC05	Registrar denúncia com foto	Visitante	Cadastra um animal em risco com foto, descrição e localização.	1
UC06	Acompanhar jornada do animal	Visitante	Consulta detalhes, progresso e histórico do animal; o visitante, só das próprias denúncias.	1
UC07	Solicitar adoção	Morador	Preenche o formulário de adoção de um animal disponível.	3
UC08	Consultar vaquinhas e copiar PIX	Morador	Consulta as vaquinhas abertas, o progresso da meta e copia a chave PIX.	2
UC09	Alterar situação do resgate	Administrador	Avança a situação do animal no fluxo de resgate, com observação.	2
UC10	Editar dados do animal	Administrador	Atualiza as informações cadastrais do animal.	1
UC11	Consultar avisos	Visitante	Vê e marca como lidos os avisos: mudança de situação das próprias denúncias e pedidos avaliados; o administrador recebe também denúncias e pedidos novos.	3
UC12	Excluir animal	Administrador	Remove registros indevidos (exceto adotados).	1
UC13	Gerenciar usuários	Administrador	Cadastra, edita, exclui e define perfis de usuários.	1
UC14	Prestar contas das doações	Administrador	Registra e publica a aplicação das doações.	Final
UC15	Entrar como convidado	Visitante	Usa o aplicativo sem conta, em sessão anônima.	2
UC16	Gerenciar vaquinhas	Administrador	Cria, edita, encerra, reabre e exclui campanhas de arrecadação.	2
UC17	Publicar notícias no mural	Administrador	Publica, edita, exclui e fixa notícias da ONG.	3
UC18	Avaliar pedidos de adoção	Administrador	Analisa, aprova ou recusa pedidos na fila, com o contato de quem pediu.	3
UC19	Ler mural de notícias	Morador	Lê as notícias da ONG e abre o animal ou a vaquinha vinculada.	3
UC20	Consultar mapa de ocorrências	Visitante	Vê no mapa os animais na cor da sua situação; o visitante, só as próprias denúncias.	2
UC21	Capturar localização por GPS	Visitante (via UC05)	Incluído na denúncia: captura a coordenada ao abrir o formulário.	2
UC22	Guardar denúncia sem conexão	Visitante (via UC05)	Estende a denúncia: sem internet, guarda no aparelho e envia quando a conexão volta.	2
UC23	Acompanhar meus pedidos de adoção	Morador	Consulta a situação dos próprios pedidos e a observação da ONG.	3
UC24	Lançar valor arrecadado	Administrador (via UC16)	Estende a gestão de vaquinhas: registra um valor recebido, que move a barra de progresso.	2
UC25	Registrar a adoção do animal	Administrador (via UC18)	Estende a avaliação: ao aprovar, marca o animal como adotado — a mesma mudança de situação do UC09 — e recusa os demais pedidos abertos.	3

5. Modelo do Banco de Dados

O banco de dados é um PostgreSQL gerenciado pelo Supabase. O modelo possui oito tabelas na aplicação — perfis, animais, histórico de situações, vaquinhas, lançamentos de vaquinha, notícias, pedidos de adoção e avisos — além da tabela de contas de autenticação mantida pelo próprio Supabase, à qual os perfis se ligam. As colunas de latitude e longitude são preenchidas pelo GPS no registro da denúncia. A Figura 2 mostra as tabelas e as relações principais; as marcadas como NOVA foram criadas na Entrega 3.

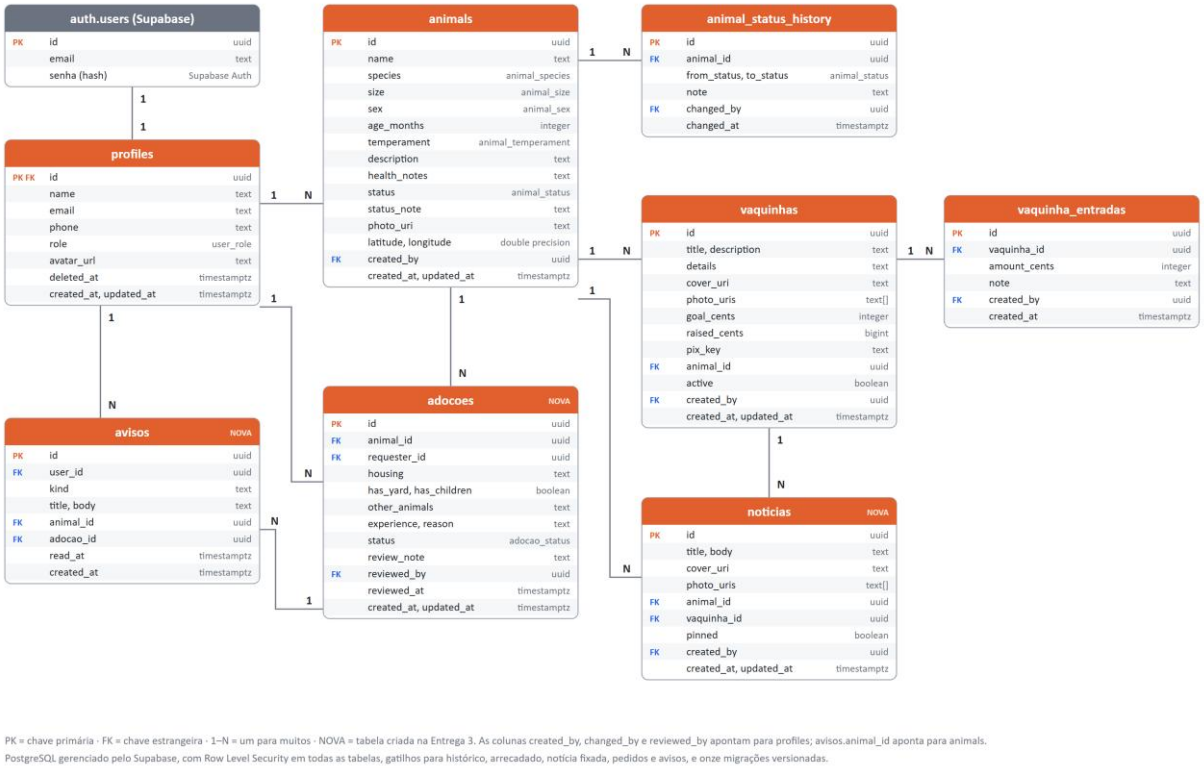


Figura 2 – Modelo entidade-relacionamento do banco de dados

Dicionário de dados

Tabela profiles – perfis dos usuários (as credenciais ficam em auth.users, sob responsabilidade do Supabase)

Campo	Tipo	Restrições	Descrição
id	uuid	PK, FK → auth.users	Identificador único (UUID).
name	text	NOT NULL	Nome completo.
email	text	UNIQUE	E-mail de acesso; nulo no visitante sem conta.
phone	text	—	Telefone com DDD, somente dígitos.
role	user_role	NOT NULL	Perfil: morador ou admin.
avatar_url	text	até 500 caracteres	URL pública da foto de perfil (nova na Entrega 3).
deleted_at	timestamptz	—	Data da exclusão lógica; nulo enquanto o perfil está ativo.
created_at / updated_at	timestamptz	NOT NULL	Datas de criação e atualização.

Tabela animals – animais denunciados, resgatados e para adoção

Campo	Tipo	Restrições	Descrição
id	uuid	PK	Identificador único (UUID).
name	text	NOT NULL	Nome do animal ("Sem nome" quando desconhecido).
species	animal_species	NOT NULL	Espécie: cachorro, gato ou outro.
size	animal_size	NOT NULL	Porte: pequeno, medio ou grande.
sex	animal_sex	NOT NULL	Sexo: macho, femea ou desconhecido.
age_months	integer	CHECK ≥ 0	Idade aproximada em meses.
temperament	animal_temperament	—	Dócil, brincalhão, tímido, agitado ou bravo.
description	text	NOT NULL	Descrição, local e condição do animal.
health_notes	text	—	Vacinas, castração, tratamentos.
status	animal_status	NOT NULL, índice	Situação atual na jornada do animal.
status_note	text	—	Observação enviada com a nova situação; o gatilho a copia para o histórico e a apaga.
photo_uri	text	—	URL pública da foto no Storage do Supabase.
latitude / longitude	double precision	—	Coordenadas capturadas por GPS no registro.
created_by	uuid	FK → profiles (ON DELETE SET NULL)	Quem registrou o animal; nulo se a conta foi excluída, preservando o registro.
created_at / updated_at	timestamptz	NOT NULL	Datas de criação e atualização.

Tabela animal_status_history – linha do tempo do animal

Campo	Tipo	Restrições	Descrição
id	uuid	PK	Identificador único (UUID).
animal_id	uuid	NOT NULL, FK → animals (cascade)	Animal ao qual a mudança se refere.
from_status	animal_status	—	Situação anterior (vazia no cadastro).
to_status	animal_status	NOT NULL	Nova situação.
note	text	—	Observação sobre a mudança.
changed_by	uuid	FK → profiles (ON DELETE SET NULL)	Responsável pela mudança; nulo se a conta foi excluída.
changed_at	timestamptz	NOT NULL, índice	Data e hora da mudança, gravada pelo servidor.

Tabela vaquinhas – campanhas de arrecadação

Campo	Tipo	Restrições	Descrição
id	uuid	PK	Identificador único (UUID).
title	text	NOT NULL, CHECK 3–80	Título da campanha.
description	text	NOT NULL, CHECK 10–1000	O que a arrecadação pretende cobrir.
details	text	até 5000 caracteres	Texto longo da página da campanha.
cover_uri	text	—	URL pública da capa no Storage.
photo_uris	text[]	NOT NULL, até 10 itens	Galeria de fotos da campanha.
goal_cents	integer	NOT NULL, CHECK > 0	Meta em centavos.
raised_cents	bigint	NOT NULL, CHECK ≥ 0	Soma dos lançamentos, mantida por gatilho.
pix_key	text	—	Chave PIX copiável; nula enquanto a ONG não informar.
animal_id	uuid	FK → animals (ON DELETE SET NULL)	Campanha de um animal específico; nula quando é da ONG em geral.
active	boolean	NOT NULL, índice	Campanha aberta ou encerrada.

Campo	Tipo	Restrições	Descrição
created_by	uuid	FK → profiles (ON DELETE SET NULL)	Administrador que criou a campanha.
created_at / updated_at	timestamptz	NOT NULL	Datas de criação e atualização.

Tabela vaquinha_entradas – lançamentos de arrecadação

Campo	Tipo	Restrições	Descrição
id	uuid	PK	Identificador único (UUID).
vaquinha_id	uuid	NOT NULL, FK → vaquinhas (cascade), índice	Campanha do lançamento.
amount_cents	integer	NOT NULL, CHECK > 0	Valor recebido, em centavos.
note	text	até 200 caracteres	Observação (ex.: "Bazar de sábado").
created_by	uuid	FK → profiles (ON DELETE SET NULL)	Administrador que lançou o valor.
created_at	timestamptz	NOT NULL	Data do lançamento, gravada pelo servidor.

Tabela noticias – mural de notícias da ONG (nova na Entrega 3)

Campo	Tipo	Restrições	Descrição
id	uuid	PK	Identificador único (UUID).
title	text	NOT NULL, CHECK 3–100	Título da notícia.
body	text	NOT NULL, CHECK 10–5000	Texto da notícia.
cover_uri	text	—	URL pública da capa no Storage.
photo_uris	text[]	NOT NULL, até 6 itens	Galeria de fotos.
animal_id	uuid	FK → animals (ON DELETE SET NULL)	Animal vinculado à notícia.
vaquinha_id	uuid	FK → vaquinhas (ON DELETE SET NULL)	Vaquinha vinculada; no máximo um dos dois vínculos (CHECK).
pinned	boolean	NOT NULL, único quando verdadeiro	Notícia fixada no topo; fixar outra desafixa a anterior (gatilho).
created_by	uuid	FK → profiles (ON DELETE SET NULL)	Administrador que publicou.
created_at / updated_at	timestamptz	NOT NULL	Datas de criação e atualização.

Tabela adocoes – pedidos de adoção (nova na Entrega 3)

Campo	Tipo	Restrições	Descrição
id	uuid	PK	Identificador único (UUID).
animal_id	uuid	NOT NULL, FK → animals (cascade)	Animal pedido; precisa estar disponível (gatilho).
requester_id	uuid	NOT NULL, FK → profiles (cascade)	Quem pediu a adoção.
housing	text	NOT NULL, CHECK casa, apartamento ou sítio	Tipo de moradia.
has_yard	boolean	NOT NULL	Se há pátio ou área cercada.
other_animals	text	até 300 caracteres	Outros animais da casa.
has_children	boolean	NOT NULL	Se moram crianças na casa.
experience	text	até 500 caracteres	Experiência com animais.
reason	text	NOT NULL, CHECK 10–1000	Motivo da adoção.
status	adocao_status	NOT NULL, índice	Enviada, em análise, aprovada ou recusada.

Campo	Tipo	Restrições	Descrição
review_note	text	até 500 caracteres	Observação da avaliação, enviada no aviso a quem pediu.
reviewed_by / reviewed_at	uuid / timestamptz	gravados por gatilho	Quem avaliou e quando.
created_at / updated_at	timestamptz	NOT NULL	Datas de criação e atualização.

Tabela avisos – central de avisos (nova na Entrega 3)

Campo	Tipo	Restrições	Descrição
id	uuid	PK	Identificador único (UUID).
user_id	uuid	NOT NULL, FK → profiles (cascade), índice	Destinatário do aviso.
kind	text	NOT NULL, CHECK	Denúncia nova, mudança de situação, pedido de adoção novo ou pedido avaliado.
title / body	text	NOT NULL	Título e texto do aviso.
animal_id	uuid	FK → animals (cascade)	Animal ao qual o aviso leva.
adocao_id	uuid	FK → adocoes (cascade)	Pedido de adoção ao qual o aviso leva.
read_at	timestamptz	—	Quando foi lido; nulo enquanto não lido.
created_at	timestamptz	NOT NULL	Data do aviso, gravada pelo servidor.

A estrutura é criada e evoluída por onze migrações SQL numeradas e versionadas junto ao código. O registro inicial do histórico e cada mudança de situação são gravados por gatilhos no banco, garantindo que nenhum animal exista sem histórico e que nenhuma mudança de situação passe sem rastro. As regras que não podem depender do aplicativo também moram no servidor: um gatilho recusa transição de situação inválida, outro impede que a ONG fique sem administrador, outro só aceita pedido de adoção para animal disponível e valida a avaliação, e um índice garante um pedido aberto por pessoa e animal. Os avisos são criados por gatilhos a cada denúncia, mudança de situação e pedido de adoção. O histórico com nome e foto de quem agiu é lido por uma função do banco (animal_history), sem abrir os perfis aos demais usuários. A exclusão de usuário é lógica e passa por uma função do banco, e a criação de contas pelo administrador é feita por uma Edge Function, única parte do sistema autorizada a usar a chave de serviço.

6. Tecnologias utilizadas

O Patinhas em Ação é desenvolvido como aplicativo multiplataforma com React Native e Expo, em TypeScript. O back-end é o Supabase, que fornece banco PostgreSQL, autenticação e armazenamento de arquivos; o Firebase, citado na proposta inicial, foi descartado por não ser relacional. As versões abaixo são as adotadas no projeto; na Entrega 3, o projeto passou do Expo SDK 57 para o 58.

Camada	Tecnologia	Versão	Finalidade
Linguagem	TypeScript	6.0	Linguagem principal, com tipagem estrita.
Front-end	React Native / React	0.88 / 19.3	Construção da interface nativa.
Front-end	Expo SDK	58	Plataforma de desenvolvimento, build e acesso a recursos nativos.
Front-end	Expo Router	58	Navegação baseada em arquivos e rotas protegidas por perfil.
Front-end	expo-image, react-native-svg, expo-symbols, expo-clipboard	58 / 15.16	Imagens, ilustrações e mascote, ícones nativos e cópia da chave PIX.
Front-end	Fontes Archivo e Chivo Mono (Google Fonts)	0.4	Tipografia da identidade visual.
Recursos nativos	expo-image-picker, expo-image-manipulator, expo-file-system, expo-location	58	Câmera, galeria, redução e envio das fotos e captura de GPS na denúncia.
Recursos nativos	expo-network, expo-background-task, expo-task-manager	58	Denúncia sem conexão: detectar a rede e enviar as pendentes, inclusive em segundo plano.
Segurança	Supabase Auth, expo-crypto, expo-sqlite	2.x / 58	Autenticação, sessão anônima do convidado, armazenamento local da sessão e geração de UUIDs.
Lógica / validação	Zod	4.6	Validação dos dados de entrada com mensagens por campo.
Banco de dados	Supabase (PostgreSQL, Auth e Storage) via supabase-js	2.116	Banco relacional remoto, com Row Level Security e migrações versionadas.
Back-end	Supabase Edge Functions (Deno)	—	Função admin-users: criação de contas com a chave de serviço, que não pode sair do servidor.
Mapas	react-native-maps, expo-location	1.29	Mapa de ocorrências com Google Maps no Android; no iOS o aplicativo usa o provedor nativo (Apple Maps).
Testes	Jest + jest-expo	30.5 / 58	Testes de regras de domínio, permissões e serviços.
Ambiente	Node.js e npm	24.16 / 12.0	Execução das ferramentas e gerenciamento de dependências.
Ambiente	Visual Studio Code	—	IDE de desenvolvimento.
Ambiente	Expo Go	SDK 58	Execução do aplicativo em dispositivos físicos durante o desenvolvimento.
Versionamento	Git e GitHub	—	Controle de versão do código-fonte.

7. Arquitetura do Software

O aplicativo segue uma arquitetura em camadas inspirada na Clean Architecture. As dependências apontam sempre para o domínio: a apresentação utiliza os serviços da aplicação, a aplicação utiliza as entidades, regras e contratos do domínio, e a infraestrutura implementa esses contratos usando os recursos do dispositivo.

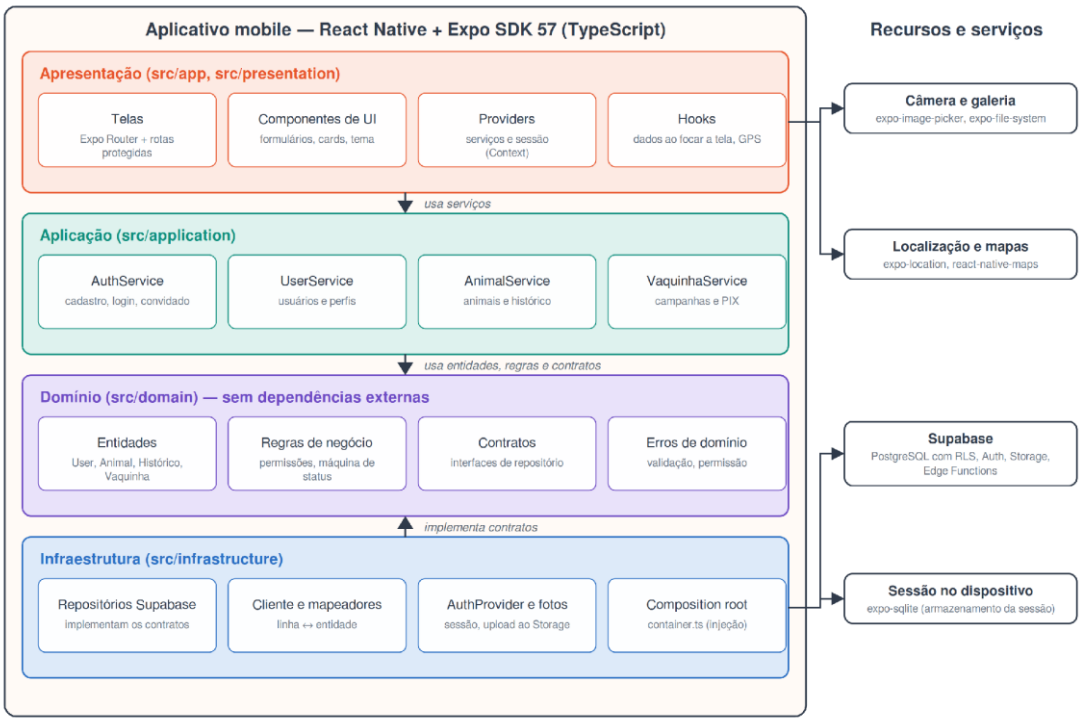


Figura 3 – Arquitetura em camadas e comunicação com recursos do dispositivo

Responsabilidades das camadas

- **Apresentação:** telas do Expo Router, componentes visuais (entre eles o mascote Caramelo, o cabeçalho das abas e o esboço de carregamento), tema e providers de contexto. As rotas são protegidas conforme a sessão e o perfil do usuário.
- **Aplicação:** serviços de autenticação, usuários, animais, vaquinhas, notícias, adoções e avisos. Validam os dados com Zod, verificam permissões e coordenam os repositórios.
- **Domínio:** entidades, máquinas de estados do resgate e do pedido de adoção, matriz de permissões, interfaces de repositório e erros de negócio, sem dependência de bibliotecas externas.
- **Infraestrutura:** repositórios Supabase, cliente e mapeamento das tabelas, provedor de autenticação, fila de denúncias sem conexão e redução e envio de fotos para o Storage. O arquivo container.ts monta e injeta as dependências (composition root).

Exemplo de comunicação: registrar uma denúncia

- A tela de registro captura a localização por GPS ao abrir e coleta os dados e a foto (câmera ou galeria); a foto é reduzida no aparelho e enviada ao Storage do Supabase, que devolve a URL pública gravada no registro. Sem internet, a denúncia e a foto ficam na fila do aparelho.
- O AnimalService verifica se o perfil pode registrar animais, valida os dados com Zod e gera os identificadores.
- O serviço chama o contrato AnimalRepository; a implementação Supabase grava o animal, e gatilhos no banco criam a primeira entrada do histórico e o aviso para a equipe da ONG no mesmo instante.
- A tela recebe o animal criado e navega para os detalhes; erros de validação retornam para os campos do formulário.

Evolução prevista

A inversão de dependência existe para que trocar o mecanismo de persistência custe pouco: repositórios novos e uma linha no composition root, sem alteração nas telas, nas regras de domínio ou nos serviços. A autenticação é a exceção prevista — senha e sessão não são persistência, e sim responsabilidade de um provedor externo, o que exige uma porta própria (AuthProvider). É um limite útil do princípio: trocar onde os dados moram é barato; trocar quem responde pela identidade não é. A Sprint 3 confirmou o desenho: notícias, adoções e avisos entraram como três serviços e três repositórios novos, sem mudança nos existentes.

8. Backlog do produto

O backlog foi priorizado a partir das funcionalidades da proposta e do planejamento de entregas. As estimativas estão em horas de desenvolvimento.

ID	Funcionalidade	Prioridade	Estimativa	Sprint
PB01	Levantamento de requisitos e modelagem (casos de uso e banco de dados)	Alta	12 h	1
PB02	Configuração do projeto e arquitetura em camadas	Alta	10 h	1
PB03	Banco de dados com migrações e dados de demonstração	Alta	8 h	1
PB04	Cadastro e autenticação de usuários (RF0001, RF0002)	Alta	10 h	1
PB05	Gerenciamento de usuários e perfil (RF0003, RF0004)	Alta	10 h	1
PB06	Registro de animais com foto (RF0005)	Alta	12 h	1
PB07	Catálogo com busca e filtros (RF0006)	Alta	8 h	1
PB08	Edição, exclusão e histórico de animais (RF0007, RF0008)	Média	8 h	1
PB09	Protótipo funcional e identidade visual	Média	14 h	1
PB10	Testes automatizados de domínio e serviços	Média	6 h	1
PB11	Documentação da Entrega 1	Alta	6 h	1
PB12	Geolocalização automática na denúncia (RF0009)	Alta	10 h	2
PB13	Mapa de ocorrências com react-native-maps (RF0010)	Média	14 h	2
PB14	Fluxo de alteração da situação do resgate (RF0011)	Alta	10 h	2
PB15	Formulário digital de adoção (RF0013)	Alta	12 h	3
PB16	Vaquinhas: meta, progresso e chave PIX (RF0014)	Média	10 h	2
PB17	Migração para o Supabase (PostgreSQL, Auth e Storage)	Média	16 h	2
PB18	Central de avisos no aplicativo (RF0012)	Média	12 h	3
PB19	Prestação de contas visual (RF0015)	Média	10 h	Final
PB20	Polimento de UI/UX e testes de usabilidade	Alta	16 h	Final
PB21	Documentação final e apresentação	Alta	8 h	Final
PB22	Melhorias na interface e identidade visual escura	Alta	14 h	2
PB23	Foto de perfil do usuário (RF0016)	Baixa	6 h	3
PB24	Mural de notícias da ONG (RF0017)	Média	12 h	3
PB25	Denúncia sem conexão com envio posterior (RF0018)	Alta	20 h	2
PB26	Lares temporários (RF0019) – fora do escopo	Alta	16 h	—
PB27	Acompanhamento pós-adoção (RF0020)	Alta	12 h	Final
PB28	Fila de casos abertos para a ONG (RF0021)	Alta	10 h	Final
PB29	Prestação de contas periódica e vaquinhas revisadas (RF0022, RF0023)	Alta	14 h	Final
PB30	Modo convidado: denúncia sem cadastro, com conversão em conta	Alta	10 h	2
PB31	Exclusão lógica de usuários e Edge Function admin-users	Média	8 h	2
PB32	Atualização para o Expo SDK 58 (React Native 0.88, Jest 30)	Média	4 h	3
PB33	Refino da interface: login com mascote, ícone do app, cadastro simplificado e ajustes de auditoria	Alta	14 h	3

9. Sprint 1

Foco: Base e Cadastro. **Período:** 03/08/2026 a 14/09/2026.

Objetivo: entregar o protótipo funcional do aplicativo, a arquitetura e o banco de dados e o cadastro completo (CRUD) de animais e usuários, criando a base para as funcionalidades de localização, adoção e doações das próximas sprints.

Planejamento

Funcionalidade (Backlog)	Responsável	Início	Término	Duração
PB01 – Requisitos e modelagem	Arthur	03/08	14/08	12 h
PB02 – Projeto e arquitetura	Arthur	10/08	17/08	10 h
PB03 – Banco de dados e migrações	Arthur	17/08	21/08	8 h
PB04 – Cadastro e autenticação	Pedro	17/08	26/08	10 h
PB05 – Usuários e perfil	Pedro	24/08	02/09	10 h
PB06 – Registro de animais com foto	Arthur	24/08	04/09	12 h
PB07 – Catálogo com busca e filtros	João	31/08	07/09	8 h
PB08 – Edição, exclusão e histórico	Pedro	31/08	09/09	8 h
PB09 – Protótipo e identidade visual	João	01/09	11/09	14 h
PB10 – Testes automatizados	Pedro	07/09	12/09	6 h
PB11 – Documentação da Entrega 1	João	08/09	14/09	6 h
Total				**104 h**

10. Sprint 2

Foco: Localização e Status. **Período:** 15/09/2026 a 05/10/2026.

Objetivo: integrar mapas e GPS, implementar o fluxo de alteração da situação do resgate, migrar a persistência para o Supabase e liberar o modo convidado e as campanhas de arrecadação. A denúncia sem conexão, prevista para a Sprint 3, foi antecipada.

Planejamento

Funcionalidade (Backlog)	Responsável	Início	Término	Duração
PB12 – Geolocalização na denúncia	Equipe	—	—	10 h
PB13 – Mapa de ocorrências	Equipe	—	—	14 h
PB14 – Alteração da situação do resgate	Equipe	—	—	10 h
PB17 – Migração para o Supabase	Equipe	—	—	16 h
PB30 – Modo convidado	Equipe	—	—	10 h
PB16 – Vaquinhas com meta e chave PIX	Equipe	—	—	10 h
PB31 – Exclusão lógica e Edge Function	Equipe	—	—	8 h
PB22 – Interface e identidade escura	Equipe	—	—	14 h
PB25 – Denúncia sem conexão	Equipe	—	—	20 h
Total				**112 h**

Entregas

- **Geolocalização (PB12):** o GPS é capturado assim que o formulário de denúncia abre e pode ser atualizado com um toque; a denúncia é aceita sem coordenada, com aviso de que não aparecerá no mapa.
- **Mapa (PB13):** é a tela inicial; mostra a localização do usuário e um marcador por animal na cor da sua situação, com cartão de foto e nome. Google Maps no Android e Apple Maps no iOS.
- **Situação do resgate (PB14):** o administrador avança a situação oferecendo só as transições válidas, com observação opcional; a regra é verificada no aplicativo e por gatilho no banco, que grava o histórico.
- **Supabase (PB17):** PostgreSQL com Row Level Security espelhando a matriz de permissões, Supabase Auth e Storage para as fotos, em seis migrações versionadas.
- **Modo convidado (PB30):** sessão anônima que registra denúncias e acompanha só as próprias; ao criar conta, a identidade é mantida e as denúncias continuam com a pessoa.
- **Vaquinhas (PB16):** capa, texto de detalhes, galeria de até 10 fotos, vínculo com um animal, chave PIX copiável e lançamentos de arrecadação que movem a barra de progresso; encerrar e reabrir campanha.
- **Exclusão lógica (PB31):** o perfil excluído some e perde o acesso, mas a autoria dos registros permanece; a Edge Function admin-users ficou só com a criação de contas.

- **Interface (PB22):** identidade escura em todas as telas, revisada em auditoria de acessibilidade (alvos de toque de 44 pt, contraste mínimo de 4,5:1 e rótulos para leitor de tela).
- **Denúncia sem conexão (PB25):** sem internet, a denúncia — com foto e coordenada — fica numa fila no aparelho, aparece como "Pendente" e é enviada sozinha quando a conexão volta, inclusive em segundo plano. O identificador gerado no aparelho impede registros duplicados.

Testes e verificação

40 testes automatizados de domínio, serviços e da fila de denúncias sem conexão (eram 26 na Sprint 1). As migrações e as regras de acesso foram verificadas em PostgreSQL com os perfis administrador, morador e convidado.

Mudanças de escopo

- PB28 – Fila de casos abertos (RF0021) constava no backlog como Sprint 2, mas não entrou no planejamento; foi remanejado para a Sprint final.
- PB25 – Denúncia sem conexão (RF0018), previsto para a Sprint 3, foi antecipado e entregue nesta sprint.
- PB26 – Lares temporários (RF0019) foi retirado do escopo do projeto.

11. Sprint 3

Foco: Interação e Social. **Período:** 06/10/2026 a 09/11/2026.

Objetivo: concluir o formulário digital de adoção e os itens de relacionamento com a comunidade — mural de notícias, central de avisos e foto de perfil. Na mesma sprint, o projeto foi atualizado para o Expo SDK 58 e a interface passou por um refino, com o login redesenhado a partir de protótipos.

Planejamento

Funcionalidade (Backlog)	Responsável	Início	Término	Duração
PB24 – Mural de notícias da ONG	Equipe	—	—	12 h
PB15 – Formulário de adoção	Equipe	—	—	12 h
PB23 – Foto de perfil do usuário	Equipe	—	—	6 h
PB18 – Central de avisos	Equipe	—	—	12 h
PB32 – Atualização para o Expo SDK 58	Equipe	—	—	4 h
PB33 – Refino da interface	Equipe	—	—	14 h
Total				**60 h**

Entregas

- **Mural de notícias (PB24):** nova aba Mural, com notícias em cartões clicáveis — título, texto, capa, até seis fotos e vínculo opcional com um animal ou uma vaquinha. O administrador publica, edita, exclui e fixa uma notícia no topo; fixar outra desafixa a anterior, regra garantida pelo banco. Quem tem conta lê; o convidado não vê. Para abrir espaço para o Mural, a gestão de usuários passou para dentro do Perfil do administrador.
- **Formulário de adoção (PB15):** botão "Quero adotar" na página do animal disponível; o morador acompanha em "Meus pedidos de adoção" e o administrador avalia em "Pedidos de adoção", com o telefone e o e-mail de quem pediu. O pedido segue Enviado → Em análise → Aprovado ou Recusado, e pode ser recusado já de início. Um pedido aberto por pessoa e animal, garantido por índice no banco. Ao aprovar, a opção "Aprovar e marcar adotado" registra a adoção no histórico do animal e recusa os demais pedidos abertos para ele, avisando cada pessoa. O catálogo passou a esconder os adotados por padrão.
- **Foto de perfil (PB23):** a foto é escolhida em Meus dados e aparece no perfil, na gestão de usuários e, com o nome, ao lado de cada ação no histórico do animal. O histórico é lido por uma função do banco que expõe só nome e foto, sem abrir os perfis aos demais usuários.
- **Central de avisos (PB18):** sino com contador de não lidos no Mapa e no Perfil e uma lista de avisos que leva ao animal ou ao pedido. Os avisos são criados por gatilhos no banco: denúncia nova e pedido de adoção novo avisam os administradores; mudança de situação avisa quem denunciou, inclusive o convidado; pedido avaliado avisa quem pediu. Notificações push continuam fora, por exigirem build própria.

- **Expo SDK 58 (PB32):** React Native 0.88, React 19.3 e todos os módulos do Expo alinhados à nova versão; Jest 30 com o preset do React Native, que passou a ser um pacote separado.
- **Refino da interface (PB33):** três propostas de login foram prototipadas e comparadas lado a lado; a escolhida apresenta o Caramelo, um mascote vira-lata caramelo que reage ao uso — olha o e-mail sendo digitado, fecha os olhos na senha, fica triste no erro e cobre a tela com sua cor ao entrar. O mascote virou também o ícone do aplicativo. O cadastro passou a pedir só nome, e-mail e senha. Uma auditoria técnica da interface orientou os demais ajustes: barra de abas respeitando a área segura do aparelho, fotos reduzidas antes do envio, teclado tratado também no Android, cabeçalho padrão nas abas, fotos de topo proporcionais à tela, largura máxima em tablets, esboço de carregamento nas listas e respeito à opção de reduzir movimento.

Testes e verificação

55 testes automatizados (eram 40 na Sprint 2), cobrindo as regras e os serviços de notícias, adoções e avisos, a máquina de estados do pedido de adoção e o atalho de aprovar e marcar adotado. As onze migrações e o fluxo completo — denúncia, mudança de situação, pedido de adoção, aprovação com registro da adoção e avisos — foram verificados em PostgreSQL com os perfis administrador, morador e convidado. O login e o cadastro foram conferidos no navegador; a validação no aparelho fica para os testes de usabilidade da Sprint final.

Validação com a ONG

Durante a sprint, as telas foram apresentadas a Vera, da ONG Patinhas em Ação, pessoalmente e por mensagem (seção 13). O retorno foi positivo: ela disse que o aplicativo "ficou muito bonito, melhor do que eu esperava" e destacou que, com as vaquinhas, ele "já está bem completo". Trouxe também uma sugestão: o aplicativo precisa funcionar sem internet no local do resgate, onde o sinal costuma faltar. A denúncia já funciona assim desde a Sprint 2 (RF0018); a sugestão reforça o teste em campo, em Arvorezinha, previsto para a Sprint final.

Mudanças de escopo

- A aprovação do pedido de adoção passou a poder registrar a adoção no mesmo passo, e o pedido passou a poder ser recusado sem análise (migração 0011). A decisão veio do uso: aprovar sem marcar o animal deixava-o à vista no catálogo como disponível.
- Os avisos foram estendidos além do previsto: pedido de adoção novo e pedido avaliado também geram aviso, e o convidado recebe os avisos das próprias denúncias.
- O cadastro foi reduzido aos campos obrigatórios; telefone e foto ficam no perfil.
- Entraram dois itens não planejados: PB32 – Atualização para o Expo SDK 58 e PB33 – Refino da interface.

12. Sprint final

Foco: Refinamento e QA. **Período:** a definir.

Objetivo: polir a interface, realizar testes de usabilidade com a ONG e implementar a prestação de contas.

Planejamento

Funcionalidade (Backlog)	Responsável	Início	Término	Duração
PB19 – Prestação de contas visual	A definir	—	—	—
PB20 – Polimento de UI/UX e usabilidade	A definir	—	—	—
PB21 – Documentação final e apresentação	A definir	—	—	—
PB27 – Acompanhamento pós-adoção	A definir	—	—	—
PB28 – Fila de casos abertos (da Sprint 2)	A definir	—	—	—
PB29 – Prestação de contas periódica	A definir	—	—	—
Total				

Detalhamento

- **Testes de usabilidade (PB20):** sessões com a ONG em Arvorezinha, incluindo o teste em campo da denúncia sem conexão sugerido na validação da Sprint 3.
- **Prestação de contas (PB19, PB29):** registrar os gastos da ONG e apresentá-los junto ao que foi arrecadado, por campanha e de forma periódica, aproveitando o modelo de lançamentos das vaquinhas.

13. Registros

Esta seção reúne os registros do trabalho realizado junto à ONG Patinhas em Ação, no município de Arvorezinha/RS, e será complementada ao longo das próximas etapas do projeto.



Figura 4 – Encontro de validação do aplicativo com Vera, da ONG Patinhas em Ação

Além do encontro, as telas do aplicativo foram enviadas por mensagem para validação. As capturas abaixo mostram o envio, o retorno da ONG e a sugestão de funcionamento sem internet no local do resgate.

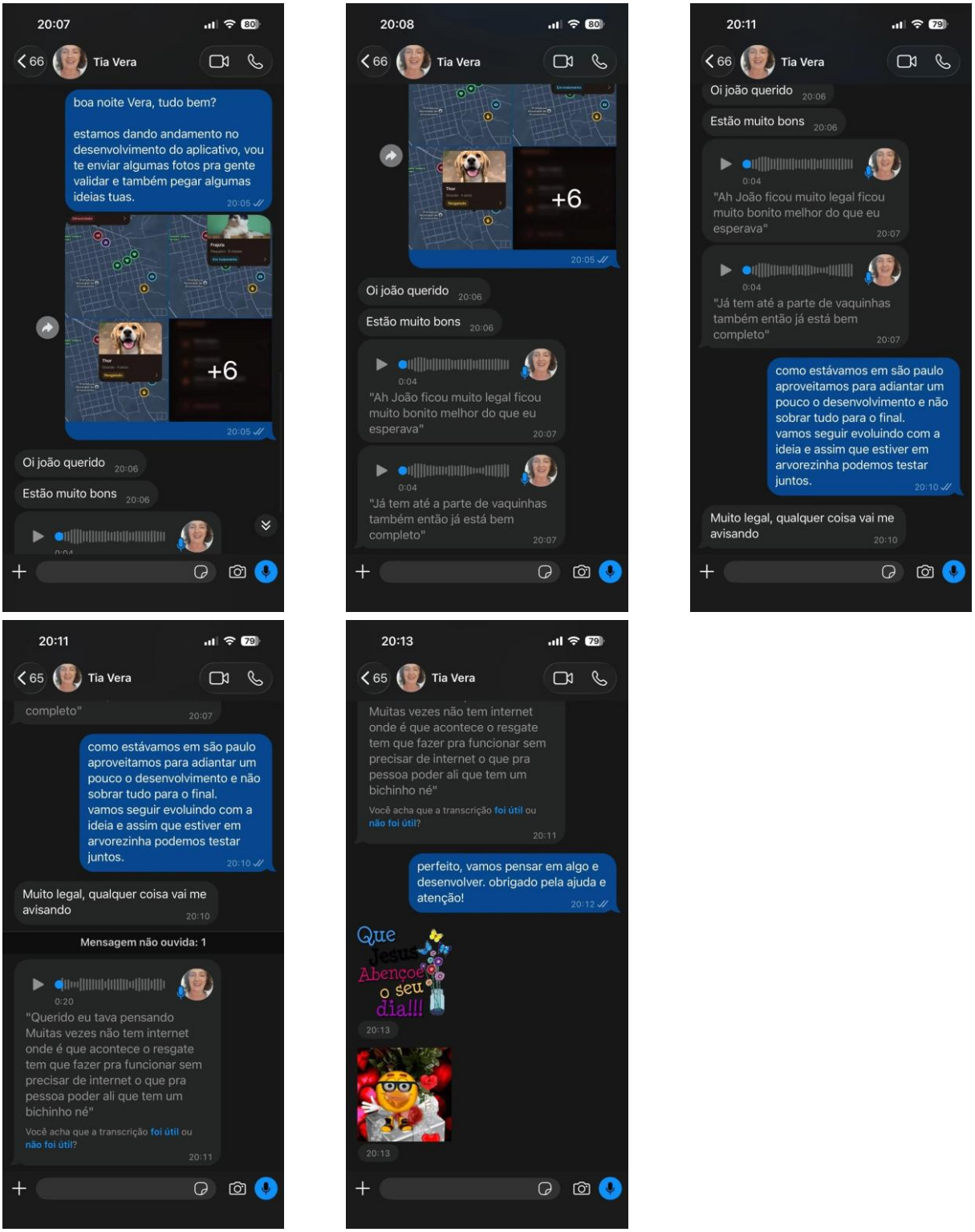


Figura 5 – Conversa de validação com a ONG: envio das telas, retorno e sugestão